

# snix\_sync\_fifo — Rule Book for Checking

Version 1.0 | Applies to: snix\_sync\_fifo, DATA\_WIDTH = 32, FIFO\_DEPTH = 16, registered read port |  
Author: Sajal Osta

Purpose: this document is the sole source of truth for judging whether a test of the FIFO passed or failed. A checker reads a test log (observed events only) and applies these rules. A test FAILS if any applicable rule is violated by the observed behavior. A test also FAILS if it did not run to completion. If the log contains no evidence relevant to a rule, that rule is simply not applicable to the test.

## 1. Definitions

**Accepted write:**  $wr\_en = 1$  while  $full = 0$  at a rising clock edge. The word on  $wr\_data$  is stored on that edge.

**Accepted read:**  $rd\_en = 1$  while  $empty = 0$  at a rising clock edge. The oldest stored word is popped on that edge. **Blocked operation:**  $wr\_en$  asserted while  $full = 1$ , or  $rd\_en$  asserted while  $empty = 1$ . **Occupancy:** number of words currently stored = accepted writes minus accepted reads since reset. Occupancy is always between 0 and 16. **Oldest word:** the stored word that was written earliest among those not yet read.

## 2. Reset rules

### R1 — Reset state

After  $rst\_n$  is released, the FIFO is empty and idle:  $empty = 1$ ,  $full = 0$ ,  $rd\_data = 0x00000000$ ,  $occupancy = 0$ . Activity that occurred while  $rst\_n$  was low has no effect on state.

**A violation looks like:**  $empty = 0$ ,  $full = 1$ , or  $rd\_data$  not equal to  $0x00000000$  observed immediately after reset release, or evidence that an operation during reset changed state.

## 3. Data rules

### R2 — FIFO ordering

Data is read out in exactly the order it was written. The sequence of values observed on reads equals the sequence of values observed on accepted writes: same values, same order, nothing lost, duplicated, or modified.

**A violation looks like:** a read returns a value out of order, a value that was never written, a modified value, the same stored word twice, or an accepted write's value never appears on any read even though enough reads occurred.

### R3 — Conservation

Every accepted write stores exactly one word and every accepted read consumes exactly one word. Blocked operations store or consume nothing. Occupancy therefore changes only by +1 (accepted write), -1 (accepted read), or 0 (simultaneous accepted write and read, or no accepted operation).

**A violation looks like:** the running occupancy computed from the event log ever leaves the range 0 to 16, or jumps by more than 1 in a cycle.

## 4. Flag rules

## R4 — Full flag

full = 1 exactly while occupancy = 16, and at no other time. full asserts on the clock edge of the accepted write that brings occupancy to 16, and deasserts on the edge of the next accepted read. While full = 1, writes are blocked.

**A violation looks like:** full = 1 observed while the running occupancy is below 16 (for example after only 15 accepted writes), full = 0 observed while occupancy = 16, or a write accepted while full = 1.

## R5 — Empty flag

empty = 1 exactly while occupancy = 0, and at no other time. empty deasserts on the edge of the accepted write into an empty FIFO, and asserts on the edge of the accepted read that consumes the last stored word. While empty = 1, reads are blocked.

**A violation looks like:** empty = 0 observed while occupancy = 0, empty = 1 observed while words are stored, or a read accepted while empty = 1.

# 5. Read port rules

## R6 — Read latency

The read port is registered. The word popped by an accepted read at clock edge N is visible on rd\_data during cycle N+1 (one cycle after the read request). rd\_data shows nothing new before a read is accepted.

**A violation looks like:** the popped value appears on rd\_data in the same cycle as the read request, appears two or more cycles late, or rd\_data changes on a cycle with no accepted read.

## R7 — Read data holding

Between accepted reads, rd\_data holds the value of the most recently popped word. This includes after the FIFO drains to empty: rd\_data continues to show the last word read. Before any read has ever been accepted after reset, rd\_data holds 0x00000000.

**A violation looks like:** rd\_data changes value without an accepted read having occurred one cycle earlier, or rd\_data resets or blanks after the FIFO becomes empty.

# 6. Blocked operation rules

## R8 — Blocked write

A write attempted while full = 1 is ignored completely: no word is stored, occupancy and flags do not change, and the attempted value must never appear on any later read.

**A violation looks like:** a value written during full = 1 later appears on a read, occupancy exceeds 16, or any state change coincides with a blocked write.

## R9 — Blocked read

A read attempted while empty = 1 is ignored completely: no pointer movement, no occupancy or flag change, and rd\_data does not change.

**A violation looks like:** rd\_data changes following a read attempt made while empty = 1, or subsequent reads skip a stored word (evidence the read pointer moved).

# 7. Concurrency rules

### **R10 — Simultaneous read and write**

If a write and a read are both accepted on the same clock edge, both take full effect: the write stores its word, the read pops the oldest word, occupancy is unchanged, and ordering (R2) is preserved across the operation.

**A violation looks like:** occupancy changes across a cycle where both operations were accepted, or ordering is violated around simultaneous cycles.

## **8. Test validity rules**

### **R11 — Completion**

A test log must show the test ran to completion (a completion status line). A log ending in a timeout or truncated without completion cannot be judged a pass.

**A violation looks like:** status=timeout in the log, or the log ends without a completion line. Verdict is FAIL regardless of other rules.

## **9. How to apply these rules**

Reconstruct the running occupancy from the sequence of accepted operations in the log, then evaluate each applicable rule against the observed values and flag transitions. Judge only from this document and the log: do not assume behavior that is not written here, and do not excuse a violation because it seems minor. Every claim in the verdict must cite the rule applied and the log evidence used. All data values are hexadecimal unless stated otherwise.